МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Организация ЭВМ и систем»

Тема: «Освоение трансляции, выполнения и отладки программ на языке Ассемблера процессора Intel x86»

Студент гр. 5387 _____ Л.А. Степанов

Преподаватель _____ А. Е. Сычев

Санкт-Петербург

2026

СОДЕРЖАНИЕ

Цель работы 3
Задание 3
Основные теоретические положения 4
Выполнение работы 6
1. Трансляция, компоновка и выполнение программы hello1 6
1.1 Подготовка среды и текст программы 6
1.2 Трансляция программы 6
1.3 Компоновка и карта памяти 7
1.4 Выполнение программы 7
1.5 Пошаговая отладка под управлением afd 8
2. Трансляция, компоновка и выполнение программы hello2 9
2.1 Структура программы и текст 9
2.2 Трансляция программы 9
2.3 Компоновка и карта памяти 10
2.4 Выполнение программы 10
2.5 Пошаговая отладка под управлением afd 10
3. Ответы на контрольные вопросы 12
3.1 Вопросы по программе hello1 12
3.2 Вопросы по программе hello2 13
Выводы 16
ПРИЛОЖЕНИЕ А 17
ПРИЛОЖЕНИЕ Б 18

Цель работы

Освоить процесс подготовки, трансляции, компоновки, выполнения и пошаговой отладки программ на языке ассемблера процессора Intel x86 в среде DOS. Разобраться в структуре программы на ассемблере, назначении сегментов, регистров процессора и в работе инструментов masm, link и отладчика afd.

Задание

Работа состоит из двух частей. В первой части необходимо подготовить учебную программу hello1.asm (упрощённое описание сегментов), изменить строку-приветствие под свои данные, протранслировать её транслятором masm с получением объектного файла и файла листинга, скомпоновать загрузочный модуль компоновщиком link с получением карты памяти, выполнить программу и выполнить её пошагово под управлением отладчика afd с фиксацией содержимого регистров до и после каждой команды. Во второй части те же действия выполняются для программы hello2.asm, в которой используется полное описание сегментов и вывод строки через процедуру.

Основные теоретические положения

Процессор выполняет только машинные команды, закодированные числами. Язык ассемблера ставит каждой машинной команде в соответствие мнемоническое обозначение, поэтому одной строке программы (кроме директив) соответствует одна команда процессора. Транслятор (masm) переводит текст программы в объектный модуль, а компоновщик (link) собирает из объектного модуля исполняемый файл, настраивая адреса сегментов.

Процессор i8086 работает в реальном режиме и адресует память по схеме «сегмент:смещение». Физический адрес вычисляется как сегмент, умноженный на 16, плюс смещение, что позволяет 16-разрядными регистрами адресовать до 1 Мбайт памяти. Базовые адреса сегментов хранятся в сегментных регистрах: CS — сегмент кода (пара CS:IP указывает на текущую команду), DS — сегмент данных, SS — сегмент стека (пара SS:SP указывает на вершину стека), ES — дополнительный сегмент.

Основную работу процессор ведёт через регистры общего назначения AX, BX, CX, DX (каждый делится на старший и младший байты, например AX = AH и AL), а также SI, DI, BP, SP. Регистр флагов хранит признаки результата операций.

Вывод на экран и завершение программы выполняет операционная система через программное прерывание int 21h. Номер требуемой функции задаётся в регистре AH. Функция 09h выводит на экран строку, адрес которой задан парой DS:DX и которая заканчивается символом «$». Функция 4Ch завершает программу. Поскольку при загрузке EXE-программы регистр DS указывает на префикс программного сегмента (PSP), а не на сегмент данных программы, первыми командами обычно выполняется настройка DS на сегмент данных. Загрузить константу непосредственно в сегментный регистр нельзя, поэтому настройка выполняется через регистр общего назначения (mov ax, @data; mov ds, ax).

PSP — служебный блок размером 256 байт, который DOS создаёт перед программой. В его начале (по смещению 0000h) размещается команда int 20h завершения программы, а в слове по смещению 0002h хранится сегментный адрес первого байта за пределами выделенной программе памяти.

Выполнение работы

Примечание. Конкретные шестнадцатеричные значения адресов сегментов в карте памяти и содержимого регистров в отладчике зависят от версии DOS и объёма памяти и определяются только в процессе загрузки программы. Приведённые ниже значения получены на конкретном прогоне в DOSBox; при выполнении работы их следует заменить значениями своего прогона. Относительные величины (длины сегментов, смещения, шестнадцатеричные коды команд, приращения IP) от машины не зависят.

1. Трансляция, компоновка и выполнение программы hello1

1.1 Подготовка среды и текст программы

Для запуска исполняемых файлов DOS использовался эмулятор DOSBox. Каталог с инструментами masm.exe, link.exe и отладчиком afd, а также с файлами hello1.asm и hello2.asm, был примонтирован как диск C: и объявлен рабочим каталогом \MASM. Строка-приветствие в программе изменена в соответствии с личными данными. Так как отладчик afd и DOSBox без дополнительной настройки не отображают кириллицу, текст приветствия записан латиницей.

Программа hello1.asm использует упрощённое описание сегментов при помощи директив .MODEL, .STACK, .DATA и .CODE. Директива DOSSEG задаёт стандартный для DOS порядок следования сегментов. Модель памяти SMALL объединяет сегмент данных и сегмент стека в одну группу DGROUP. В сегменте данных директивой DB описана строка приветствия, заканчивающаяся символами перевода строки (13, 10) и признаком конца «$». В сегменте кода первые две команды настраивают DS на сегмент данных, затем в DX загружается смещение строки, и функцией 09h прерывания 21h строка выводится на экран; завершается программа функцией 4Ch. Полный текст программы приведён в приложении А.

1.2 Трансляция программы

Трансляция выполнена командой:

```
C:\MASM> masm hello1.asm ;
```

В результате получены объектный файл hello1.obj и файл листинга hello1.lst. Синтаксических ошибок транслятор не обнаружил (0 Warning Errors, 0 Severe Errors). Файл листинга содержит исходный текст программы с номерами строк, относительные адреса (смещения) команд и данных внутри сегментов, объектный код каждой команды в шестнадцатеричном виде, таблицу сегментов и групп, таблицу имён (символов) с их типами и значениями, а также итоговое число предупреждений и ошибок. Фрагмент листинга с сегментом кода приведён ниже.

```
0000                    .CODE
0000  B8 ---- R         mov  ax,@data
0003  8E D8             mov  ds,ax
0005  BA 0000 R         mov  dx,OFFSET Greeting
0008  B4 09             mov  ah,9
000A  CD 21             int  21h
000C  B4 4C             mov  ah,4ch
000E  CD 21             int  21h
```

1.3 Компоновка и карта памяти

Компоновка выполнена командой:

```
C:\MASM> link hello1.obj ;
```

Получены загрузочный модуль hello1.exe и файл карты памяти hello1.map. Содержимое карты памяти приведено в таблице 1.

Таблица 1 – Карта памяти программы hello1

Start	Stop	Length	Name	Class
00000H	0000FH	00010H	_TEXT	CODE
00010H	00039H	0002AH	_DATA	DATA
00040H	0013FH	00100H	STACK	STACK

Из карты памяти видно, что сегмент кода _TEXT имеет длину 0010h (16 байт), сегмент данных _DATA — 002Ah (42 байта), сегмент стека STACK — 0100h (256 байт). Сегменты данных и стека объединены в группу DGROUP (модель SMALL). Точка входа программы находится по адресу 0000:0000 относительно начала сегмента кода. Таким образом, каждый сегмент размещается компактно, а суммарный объём программы невелик.

1.4 Выполнение программы

Программа выполнена в автоматическом режиме командой:

```
C:\MASM> hello1.exe
```

На экран выведена строка приветствия «Vas privetstvuet Stepanov L.A. gr. 5387», после чего управление возвращено в DOS. Программа работоспособна.

1.5 Пошаговая отладка под управлением afd

Отладка выполнена командой:

```
C:\MASM> afd hello1.exe
```

В отладчике обычные команды выполняются пошагово по клавише F1, а команды прерывания int 21h — по клавише F2, чтобы не входить внутрь обработчика прерывания. Содержимое всех регистров процессора перед выполнением первой команды приведено ниже.

```
AX=0000  BX=0000  CX=0010  DX=0000
SP=012A  BP=0000  SI=0000  DI=0000
DS=2D75  ES=2D75  SS=2D86  CS=2D85  IP=0000
NV UP EI PL NZ NA PO NC
```

При загрузке EXE-программы регистры DS и ES указывают на PSP (2D75), пара CS:IP указывает на точку входа (2D85:0000), пара SS:SP — на вершину стека. Сегмент данных программы (@data) расположен по адресу 2D86 и в регистр DS ещё не загружен. Результаты пошагового выполнения приведены в таблице 2 (звёздочкой отмечены изменившиеся значения).

Таблица 2 – Пошаговое выполнение программы hello1

Адрес	Символьный код	16-ричный код	До выполнения	После выполнения
0000	mov ax,@data	B8 86 2D	(ax)= 0000 (ip)= 0000	*(ax)= 2D86 *(ip)= 0003
0003	mov ds,ax	8E D8	(ax)= 2D86 (ds)= 2D75 (ip)= 0003	(ax)= 2D86 *(ds)= 2D86 *(ip)= 0005
0005	mov dx,OFFSET Greeting	BA 00 00	(dx)= 0000 (ip)= 0005	(dx)= 0000 *(ip)= 0008
0008	mov ah,9	B4 09	(ax)= 2D86 (ip)= 0008	*(ax)= 0986 *(ip)= 000A
000A	int 21h (F2)	CD 21	(ax)= 0986 (dx)= 0000 (ds)= 2D86 (ip)= 000A	*(ax)= 0924 *(ip)= 000C вывод строки на экран
000C	mov ah,4ch	B4 4C	(ax)= 0924 (ip)= 000C	*(ax)= 4C24 *(ip)= 000E
000E	int 21h (F2)	CD 21	(ax)= 4C24 (ip)= 000E	завершение программы, возврат в DOS

При выполнении команды mov ds,ax содержимое AX (2D86) копируется в DS, и регистр данных начинает указывать на сегмент данных программы. Команда mov ah,9 записывает 09h только в старший байт AX, не изменяя младший (2D86 переходит в 0986). После вызова функции 09h в AL остаётся код символа «$» (24h). Команды прерывания выполнялись по клавише F2.

2. Трансляция, компоновка и выполнение программы hello2

2.1 Структура программы и текст

Программа hello2.asm использует полное описание сегментов: сегменты стека (AStack), данных (DATA) и кода (CODE) описаны вручную директивами SEGMENT и ENDS. Директива ASSUME сообщает транслятору, какой сегмент соответствует каждому сегментному регистру (CS:CODE, DS:DATA, SS:AStack); сама она объектного кода не порождает, а нужна для правильного вычисления смещений при трансляции. В сегменте данных описаны две строки приветствия. Вывод строки вынесен в отдельную процедуру WriteMsg, которая ожидает смещение строки в регистре DX и вызывается командой call. Головная процедура Main объявлена как FAR. Полный текст программы приведён в приложении Б.

В начале головной процедуры в стек помещаются сегментный адрес PSP (командой push ds) и нулевое смещение (командами sub ax,ax и push ax). Это подготавливает адрес PSP:0000 для возврата в DOS: команда ret головной процедуры, будучи дальней, снимает со стека это смещение и сегмент и передаёт управление на начало PSP, где расположена команда завершения программы.

2.2 Трансляция программы

Трансляция выполнена командой masm hello2.asm ; с получением объектного файла hello2.obj и файла листинга hello2.lst. Синтаксических ошибок транслятор не обнаружил. Фрагмент листинга с сегментом кода приведён ниже.

```
0000                    CODE    SEGMENT
0000                    WriteMsg PROC NEAR
```

```
0000  B4 09             mov  AH,9
0002  CD 21             int  21h
0004  C3                ret
0005                    WriteMsg ENDP
0005                    Main   PROC FAR
0005  1E                push DS
0006  2B C0             sub  AX,AX
0008  50                push AX
0009  B8 ---- R          mov  AX,DATA
000C  8E D8             mov  DS,AX
000E  BA 0000 R          mov  DX,OFFSET HELLO
0011  E8 FFEC           call WriteMsg
0014  BA 0010 R          mov  DX,OFFSET GREETING
0017  E8 FFE6           call WriteMsg
001A  CB                ret
001B                    Main   ENDP
```

2.3 Компоновка и карта памяти

Компоновка выполнена командой link hello2.obj ; с получением загрузочного модуля hello2.exe и карты памяти hello2.map (таблица 3).

Таблица 3 – Карта памяти программы hello2

Start	Stop	Length	Name	Class
00000H	00017H	00018H	ASTACK	STACK
00020H	0005BH	0003CH	DATA	DATA
00060H	0007AH	0001BH	CODE	CODE

Сегмент стека ASTACK имеет длину 0018h (24 байта, 12 слов), сегмент данных DATA — 003Ch (60 байт), сегмент кода CODE — 001Bh (27 байт). Так как модель памяти не задавалась, сегменты не объединяются в группу и размещаются последовательно с выравниванием на границу параграфа. Точка входа программы соответствует процедуре Main и находится по смещению 0005h в сегменте кода (перед Main в сегменте расположена процедура WriteMsg).

2.4 Выполнение программы

Программа выполнена командой hello2.exe. На экран последовательно выведены две строки приветствия, после чего управление возвращено в DOS. Программа работоспособна.

2.5 Пошаговая отладка под управлением afd

Отладка выполнена командой afd hello2.exe. Содержимое всех регистров процессора перед выполнением первой команды приведено ниже.

```
AX=0000  BX=0000  CX=001B  DX=0000
SP=0018  BP=0000  SI=0000  DI=0000
```

```
DS=2D75  ES=2D75  SS=2D85  CS=2D8B  IP=0005
NV UP EI PL NZ NA PO NC
```

Регистры DS и ES указывают на PSP (2D75), сегмент стека SS равен 2D85, сегмент кода CS равен 2D8B, а точка входа IP равна 0005h, так как процедура Main расположена в сегменте кода после процедуры WriteMsg. Результаты пошагового выполнения приведены в таблице 4.

Таблица 4 – Пошаговое выполнение программы hello2

Адрес	Символьный код	16-ричный код	До выполнения	После выполнения
0005	push DS	1E	(sp)= 0018 (ds)= 2D75 (ip)= 0005	*(sp)= 0016 *(ip)= 0006 в стек: 2D75
0006	sub AX,AX	2B C0	(ax)= 0000 (ip)= 0006	(ax)= 0000 ZF=1 *(ip)= 0008
0008	push AX	50	(sp)= 0016 (ax)= 0000 (ip)= 0008	*(sp)= 0014 *(ip)= 0009 в стек: 0000
0009	mov AX,DATA	B8 87 2D	(ax)= 0000 (ip)= 0009	*(ax)= 2D87 *(ip)= 000C
000C	mov DS,AX	8E D8	(ax)= 2D87 (ds)= 2D75 (ip)= 000C	*(ds)= 2D87 *(ip)= 000E
000E	mov DX,OFFSET HELLO	BA 00 00	(dx)= 0000 (ip)= 000E	(dx)= 0000 *(ip)= 0011
0011	call WriteMsg	E8 EC FF	(sp)= 0014 (ip)= 0011	*(sp)= 0012 *(ip)= 0000 в стек: 0014
0000	mov AH,9	B4 09	(ax)= 2D87 (ip)= 0000	*(ax)= 0987 *(ip)= 0002
0002	int 21h (F2)	CD 21	(ax)= 0987 (ip)= 0002	*(ax)= 0924 *(ip)= 0004 вывод 1-й строки
0004	ret	C3	(sp)= 0012 (ip)= 0004	*(sp)= 0014 *(ip)= 0014 возврат в Main
0014	mov DX,OFFSET GREETING	BA 10 00	(dx)= 0000 (ip)= 0014	*(dx)= 0010 *(ip)= 0017
0017	call WriteMsg	E8 E6 FF	(sp)= 0014 (ip)= 0017	*(sp)= 0012 *(ip)= 0000 в стек: 001A
001A	ret (дальний)	CB	(sp)= 0014 (ip)= 001A	*(ip)= 0000 *(cs)= 2D75 переход на PSP:0000

Команда call WriteMsg (ближний вызов) помещает в стек адрес возврата — смещение следующей за ней команды (одно слово). Команда ret процедуры WriteMsg снимает это слово со стека и возвращает управление в головную процедуру. Дальняя команда ret процедуры Main снимает со стека смещение 0000 и сегмент PSP, помещённые в начале, и передаёт управление на PSP:0000, где расположена команда завершения программы. Команды прерывания выполнялись по клавише F2.

3. Ответы на контрольные вопросы

3.1 Вопросы по программе hello1

а) Какие сведения содержит файл листинга. Файл листинга содержит исходный текст программы с нумерацией строк, относительные адреса (смещения) команд и данных внутри сегментов, объектный код каждой команды в шестнадцатеричном виде, таблицу сегментов и групп с их размерами и классами, таблицу имён (символов) с указанием их типа и значения, а также итоговое количество ошибок и предупреждений трансляции.

б) Каково размещение сегментов программы в памяти. По карте памяти (таблица 1): сегмент кода _TEXT занимает адреса 0000h–000Fh (длина 16 байт), сегмент данных _DATA — 0010h–0039h (42 байта), сегмент стека STACK — 0040h–013Fh (256 байт). Сегменты данных и стека объединены в группу DGROUP. Физически при прогоне сегмент кода размещён с адреса 2D85, сегмент данных (DGROUP) — с адреса 2D86.

в) Что будет, если убрать первые две команды. Команды mov ax,@data и mov ds,ax настраивают DS на сегмент данных. Если их убрать, регистр DS останется указывать на PSP (2D75). Команда mov dx,OFFSET Greeting загрузит правильное смещение, но функция 09h будет читать байты по адресу PSP:0000, то есть содержимое начала PSP. На экран выведется произвольный набор символов до первого встретившегося байта 24h («$»), а не строка приветствия. Программа при этом завершится нормально.

г) Достаточно ли памяти под сегмент кода. По отладке сегмент кода размещается с адреса CS = 2D85, а сегмент данных начинается с адреса 2D86, то есть под код отведён один параграф (16 байт). Длина сегмента кода по карте памяти равна 0010h = 16 байт, поэтому код полностью помещается в отведённую область. Кроме того, DOS по умолчанию выделяет EXE-программе всю доступную память: слово PSP по смещению 0002h содержит сегмент конца выделенного блока (на прогоне — значение вида 9FFFh), что значительно превышает адреса, занимаемые кодом.

д) Достаточно ли памяти под сегмент данных. Сегмент данных (DGROUP) размещается с адреса 2D86, его длина по карте памяти составляет 002Ah (данные) плюс стек. Первый байт за пределами выделенной программе памяти определяется по слову PSP:0002 (на прогоне — 9FFFh) и располагается намного дальше конца сегмента данных, поэтому выделенной памяти заведомо достаточно. Проверка выполнена по результатам отладки, так как память выделяется только при загрузке программы.

3.2 Вопросы по программе hello2

а) Каково размещение сегментов программы в памяти. По карте памяти (таблица 3): сегмент стека ASTACK — 0000h–0017h (24 байта), сегмент данных DATA — 0020h–005Bh (60 байт), сегмент кода CODE — 0060h–007Ah (27 байт). Сегменты в группу не объединяются (модель памяти не задана) и следуют с выравниванием на границу параграфа. Физически при прогоне SS = 2D85, DS (после настройки) = 2D87, CS = 2D8B.

б) Что будет, если убрать директиву ASSUME (полностью и частично). ASSUME — директива транслятора, которая сообщает, какому сегменту соответствует каждый сегментный регистр; собственного объектного кода она не порождает. Если убрать её полностью, транслятор не сможет правильно сопоставить обращения к данным и командам с сегментными регистрами и выдаст ошибки трансляции (обращения к меткам данных и к коду станут неопределёнными), то есть объектный модуль получить не удастся. Если убрать её частично, ошибки возникнут только для тех регистров, привязка которых снята: например, без ASSUME DS:DATA неверно вычисляются смещения и обращения к строкам, без ASSUME CS:CODE — обращения внутри сегмента кода. До выполнения программы дело не дойдёт, пока директива не будет восстановлена.

в) Что будет, если убрать 4-ю и 5-ю команды головной процедуры. Четвёртая и пятая команды (mov AX,DATA и mov DS,AX) настраивают DS на сегмент данных. Если их убрать, DS останется указывать на PSP, и функция

09h будет выводить содержимое начала PSP вместо строк приветствия — на экран попадёт произвольный набор символов (аналогично вопросу «в» по hello1). Возврат в DOS при этом отработает корректно, так как команды сохранения адреса PSP в стеке остаются на месте.

г) Что будет, если убрать первые три команды головной процедуры. Первые три команды (push DS, sub AX,AX, push AX) сохраняют в стеке адрес PSP:0000 для возврата в DOS. Если их убрать, дальняя команда ret головной процедуры снимет со стека не адрес PSP:0000, а произвольные данные, оставшиеся в стеке, и передаст управление по неверному адресу. Программа выведет строки приветствия, но при завершении произойдёт переход по случайному адресу — «зависание» или аварийное завершение. Такой способ возврата корректен только вместе с командой mov AX,4C00h / int 21h, если заменить им ret.

д) Достаточно ли памяти под сегмент стека. Сегмент стека ASTACK по карте памяти имеет длину 0018h (24 байта), при прогоне SS = 2D85, SP = 0018h. Программа помещает в стек не более нескольких слов (два слова в начале Main и адрес возврата при call), поэтому 24 байт достаточно. Кроме того, слово PSP:0002 (на прогоне 9FFFh) показывает, что выделенный DOS блок памяти простирается далеко за пределы стека. Проверка выполнена по результатам отладки.

е) Достаточно ли памяти под сегмент данных. Сегмент данных DATA размещён с адреса 2D87, его длина по карте памяти равна 003Ch (60 байт). Первый байт за границей выделенной памяти (слово PSP:0002, на прогоне 9FFFh) находится намного дальше конца сегмента данных, поэтому памяти достаточно с большим запасом. Проверка выполнена по результатам отладки.

ё) Что сохраняется в стеке при выполнении оператора call WriteMsg. Так как процедура WriteMsg объявлена как NEAR (ближняя), команда call помещает в стек адрес возврата в виде одного слова — смещение команды, следующей за call (для первого вызова это смещение 0014h). Команда ret

процедуры снимает это слово и возвращает управление по сохранённому смещению.

ж) Как выполняется возврат в DOS по оператору ret головной процедуры. Головная процедура Main объявлена как FAR, поэтому её команда ret является дальней (retf, код CB) и снимает со стека два слова — смещение и сегмент. В начале процедуры в стек были помещены сегмент PSP (push DS) и нулевое смещение (push AX после sub AX,AX). Дальний ret снимает эти значения и передаёт управление по адресу PSP:0000, где DOS заранее разместила команду int 20h завершения программы. Так осуществляется возврат в DOS.

Выводы

В ходе работы освоен полный цикл подготовки программы на языке ассемблера в среде DOS: редактирование исходного текста, трансляция транслятором masm с анализом файла листинга, компоновка компоновщиком link с анализом карты памяти, выполнение программы и пошаговая отладка под управлением отладчика afd. На примере двух программ разобрано упрощённое и полное описание сегментов, назначение сегментных регистров и группы DGROUP, а также вынос вывода строки в отдельную процедуру.

Пошаговая отладка позволила наглядно проследить настройку регистра DS на сегмент данных, изменение отдельных байтов регистров, работу стека при выполнении команд push, call и ret, а также механизм возврата в DOS через адрес PSP:0000. По карте памяти и по содержимому PSP подтверждено, что DOS выделяет программе достаточный объём памяти для размещения всех её сегментов. Цель работы достигнута.

ПРИЛОЖЕНИЕ А

Текст программы hello1.asm

```
; HELLO1.ASM -  упрощенная версия учебной программы лаб.раб. N1
; Назначение: вывод строки-приветствия функцией DOS 09h (int 21h)

   DOSSEG                           ; задание сегментов под DOS
   .MODEL  SMALL                    ; модель памяти SMALL (малая)
   .STACK  100h                     ; отвести под стек 256 байт
   .DATA                            ; начало сегмента данных
Greeting  LABEL  BYTE               ; текст приветствия
   DB 'Vas privetstvuet Stepanov L.A. gr. 5387',13,10,'$'
   .CODE                            ; начало сегмента кода
   mov  ax,@data                    ; загрузка в DS адреса начала
   mov  ds,ax                       ; сегмента данных
   mov  dx,OFFSET Greeting          ; смещение строки приветствия
DisplayGreeting:
   mov  ah,9                        ; функция DOS печати строки
   int  21h                         ; вывод строки на экран
   mov  ah,4ch                      ; функция DOS завершения
   int  21h                         ; завершение программы
   END
```

ПРИЛОЖЕНИЕ Б

Текст программы hello2.asm

```
; HELLO2.ASM - учебная программа N2, вывод строки через процедуру

EOFLine  EQU  '$'                  ; символьная константа конца строки

AStack    SEGMENT  STACK           ; сегмент стека
          DW 12 DUP(?)             ; 12 слов памяти
AStack    ENDS

DATA      SEGMENT                  ; сегмент данных
HELLO     DB 'Zdravstvuyte!', 0AH, 0DH, EOFLine
GREETING  DB 'Vas privetstvuet st.gr.5387 - Stepanov L.A.$'
DATA      ENDS

CODE      SEGMENT                  ; сегмент кода
          ASSUME CS:CODE, DS:DATA, SS:AStack

WriteMsg  PROC  NEAR               ; процедура печати строки
          mov   AH,9
          int   21h
          ret
WriteMsg  ENDP

Main      PROC  FAR                ; головная процедура
          push  DS                 ; сохранить адрес PSP
          sub   AX,AX             ; AX = 0 (смещение)
          push  AX                 ; ... для возврата по ret
          mov   AX,DATA            ; настройка сегмента
          mov   DS,AX              ; данных
          mov   DX, OFFSET HELLO   ; вывод первой строки
          call  WriteMsg
          mov   DX, OFFSET GREETING; вывод второй строки
          call  WriteMsg
          ret                      ; возврат в DOS через PSP:0000
Main      ENDP
CODE      ENDS
          END Main
```